

When the count reaches above ten (500ms time is elapsed), ISR again and check state flag. If state is Alive, it means that the program is running correctly. Otherwise, set state as Unknown. This represents that there is some problem in the execution of the code and so ISR will not kick the watchdog anymore and the system will restart after watchdog time-out of 100ms.

```

-----CODE-----
ISR() //50ms free running
{
    Count++;
    If (Count > 10) //10x50ms
    {
        Count = 0;

        If (State == ALIVE)
        {
            State = RESET;
        }
        else
        {
            State = UNKNOWN;
        }
    }

    If (State != UNKNOWN)
    {
        Kick the watchdog
    }
}
-----CODE-----

```

Never kick the watchdog in an ISR unconditionally or devote an RTOS task to this activity, because, if the main code crashes, interrupts (and even the scheduler), it may continue to run so the watchdog never times-out. However, this approach is not recommended as we have no idea if the code is working, or not, except the timer ISR.

Implementation of watchdog timer for RTOS based application

In a multitasking environment, there are a couple of independent loops running in parallel, known as tasks. The scheduler schedules each task based on priority. To validate that each task is running properly, each task must contribute in the decision

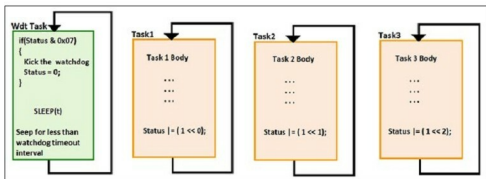


Fig. 4: Watchdog design for RTOS, approach 1

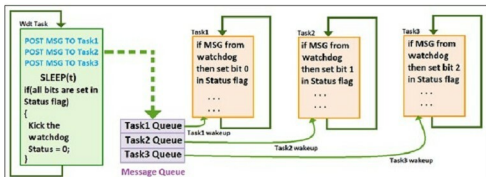


Fig. 5: Watchdog design for RTOS, approach 2

of kicking the watchdog.

To implement the watchdog mechanism in an RTOS environment, we can design a separate task that will monitor the status of all running tasks—we can call this the watchdog task. Only this task gets the privilege of kicking the watchdog.

Let us take an approach in which there is a status byte and each bit of this byte is associated with a task. For example, our system has three tasks running and each task will set corresponding bits in the status flag at the end of its body.

When the watchdog task wakes up, it will check whether all three bits are set (which means whether all tasks are running properly). It will kick the watchdog and clear the status flag. In this case, the priority of the watchdog task must be lower than other system tasks. Once the watchdog timer execution is completed, it goes in sleep mode for less than the watchdog time-out period.

The approach for the watchdog design for an RTOS (Fig. 4) will work well if all tasks are executed once in less time than the watchdog reset pe-

riod, including watchdog task. But if any of the tasks go in sleep mode for a couple of seconds, or have to wait for an event, the above approach will not work in this design.

We can implement it in a better way by using the message queue, where each task blocks at the message queue. The watchdog task will post messages to all tasks and go in sleep mode for a specified time interval (less than the watchdog time-out period).

After the arrival of the message in the message queue, the task will wake up one by one based on priority. Each task reads the message and if the task has been woken up by the watchdog task, it will set the corresponding bit in the status flag.

When the watchdog task wakes up, check the status flag. If it has all corresponding bits set, kick the watchdog and clear the status flag. In this approach, the watchdog task must have higher priority than all other system tasks.

The selection of priority of the watchdog task is very important as it depends on the design architecture of the system. ●